always has support for several SIM cards (usually, two). CPU performance is not too great, so it runs a simple OS that's designed by Mentor Graphics.

It's hard to be specific about Chinese mobile operators and their 2G/3G data and voice tariff plans, but judging by the number of dual-SIM phones available in local stores, it seems that price wars are in full swing. In addition, Chinese factories can produce their own units in almost any quantity.

You may ask, "Where is that promised smartphone? And who manufactures it?" It turns out that it's MediaTek that has won almost the entire mobile OEM market in China. After several profitable years, it wisely invested profits in R&D, and created a promising SoC named MT6516, with sufficient capability to run Windows Mobile 6.5—and later, Google's Android OS. And the zero licence charges for Android raised the question, "Why use Windows Mobile with the MT6516?" So from 2009 onwards, all smartphones based on the MT6516 were only sold with Android!

Well, let's now proceed from the lyrical side of the story to the hardware level, and see what this device's capabilities are.

Android is the same as Linux… but slimmer

After a preliminary examination, the problem was how to get under the hood and get access to internal resources? The beautiful Android UI is cool, but the underlying characteristics of this smartphone are interesting, because only by knowing about them, can you estimate the programming resources available for your future application—how fast it will perform, how much RAM it can get, and so on.

Luckily, with Android-based devices, it's easy to get a console shell (compared to the same procedure for, say, an iOS-based device like the iPhone). Just install the Android SDK on your PC, and use *adb* (the Android debugger) to connect to a shell with just an ordinary user's privileges (which is better than no access at all). Get the SDK archive [see Reference 3], unpack it, run the *android* application, and choose your Android release:

```
$ tar xjvf android-sdk_r11-linux_x86.tgz
$ cd android-sdk-linux_x86
$ ./tools/android
```

Of course, you need a JRE (Java Run-Time) pre-installed on your PC, since almost all the SDK tools are written in Java.

After launching the *android* application, correctly set the proxy field in *'Settings'*, if you connect to the Internet via a proxy. Otherwise, leave it blank. Next, in the *'Available Packages'* tab, click the *'Refresh'* button. Select the same Android release used on your phone. In my case, I needed to check *'SDK Platform Android 2.2, API 8, revision 2'*, with *'Android SDK Tools, revision 11'*, as well as *'Android SDK Platform-tools, revision 4'*. After that, click *'Install Selected'* and wait while the Android SDK environment is prepared. You can now get access to the smartphone's internals.

After connecting the phone with the USB cable, please also check that you are able to run the *tools/ddms* SDK application (Dalvik Debug Monitor System), and/or get a screenshot *(Device > Screen Capture)*. You may see errors like those listed below:

```
08:33:56 E/DDMS: insufficient permissions for device
com.android.ddmlib.AdbCommandRejectedException: insufficient
permissions for device
        at com.android.ddmlib.AdbHelper.setDevice(AdbHelper.
java:736)
        at com.android.ddmlib.AdbHelper.
executeRemoteCommand(AdbHelper.java:373)
        at com.android.ddmlib.Device.executeShellCommand(Device.
java:276)
        at com.android.ddmuilib.SysinfoPanel.
loadFromDevice(SysinfoPanel.java:159)
        at com.android.ddmuilib.SysinfoPanel.
deviceSelected(SysinfoPanel.java:126)
        at com.android.ddmuilib.SelectionDependentPanel.deviceSel
ected(SelectionDependentPanel.java:52)
        at com.android.ddms.UIThread.selectionChanged(UIThread.
java:1684)
        at com.android.ddmuilib.DevicePanel.
notifyListeners(DevicePanel.java:752)
        at com.android.ddmuilib.DevicePanel.
notifyListeners(DevicePanel.java:740)
        at com.android.ddmuilib.DevicePanel.
access$1100(DevicePanel.java:56)
        at com.android.ddmuilib.DevicePanel$1.
widgetSelected(DevicePanel.java:357)
        at org.eclipse.swt.widgets.TypedListener.
handleEvent(Unknown Source)
        at org.eclipse.swt.widgets.EventTable.sendEvent(Unknown
Source)
        at org.eclipse.swt.widgets.Widget.sendEvent(Unknown
Source)
        at org.eclipse.swt.widgets.Display.
runDeferredEvents(Unknown Source)
        at org.eclipse.swt.widgets.Display.
readAndDispatch(Unknown Source)
        at com.android.ddms.UIThread.runUI(UIThread.java:492)
        at com.android.ddms.Main.main(Main.java:103)
```

This means that the *udev* subsystem on your PC is mis-configured. To fix it, create a file */etc/udev/rules.d/53-android.rules* with the following contents:

```
SUBSYSTEM=="usb", SYSFS{idVendor}=="0bb4", MODE="0666",
```